

Memoria compartida, concurrencia y mecanismos de acceso

Memoria compartida en C bajo Linux

La memoria compartida, junto con los semáforos y las colas de mensajes, son los recursos compartidos que pone Linux a disposición de los programas para que puedan intercambiarse información.

En C para Linux es posible hacer que dos procesos (dos programas) distintos sean capaces de compartir una zona de memoria común y, de esta manera, compartir o comunicar datos.

La forma de conseguirlo en un programa es la siguiente:

- 1) En primer lugar, es necesario conseguir una clave, de tipo `key_t`, que sea común para todos los programas que quieran compartir la memoria. Para ello existe la función `key_tftok (char *, int)`. A dicha función se le pasa un fichero que exista y sea accesible y un entero. Con ellos construye una clave que devuelve. Si todos los programas utilizan el mismo fichero y el mismo entero, obtendrán la misma clave. Es habitual, como primer parámetro, pasar algún fichero del sistema que se esté seguro de su existencia, como por ejemplo `"/bin/lis"`, que es el `"ls"` del unix. Para el entero, bastaría con poner un `#define` en algún fichero.h de forma que todos los programas que vayan a utilizar la memoria compartida incluyan dicho fichero y utilicen como entero el del `#define`.
- 2) Una vez obtenida la clave, se crea la zona de memoria. Para ello está la función `intshmget (key_t, int, int)`. Con dicha función se crea la memoria y devuelve un identificador para dicha zona. Si la zona de memoria correspondiente a la Clave `key_t` ya estuviera creada, simplemente daría el identificador de la memoria (siempre y cuando los parámetros no indiquen lo contrario). El primer parámetro es la clave `key_t` obtenida anteriormente y que debería ser la misma para todos los programas. El segundo parámetro es el tamaño en bytes que se desea para la memoria.
- 3) El tercer parámetro son unos *flags*. Aunque hay más posibilidades, lo imprescindible es:
 - 9 bits menos significativos, son permisos de lectura/escritura/ejecución para propietario/grupo/otros, al igual que los ficheros. Para obtener una memoria

con todos los permisos para todo el mundo, se debe poner como parte de los *flags* el número 0777. Es importante el cero delante, para que el número se interprete en octal y queden los bits en su sitio (En C, cualquier número que empiece por cero, se considera octal). El de ejecución no tiene sentido y se ignora.

- IPC_CREAT. Junto con los bits anteriores, este bit indica si se debe crear la memoria en caso de que no exista. Si está puesto, la memoria se creará si no lo está ya y se devolverá el identificador. Si no está puesto, se intentará obtener el identificador y se obtendrá un error si no está ya creada. En resumen, los *flags* deberían ser algo así como 0777 | IPC_CREAT.

4) El último paso poder usar la memoria consiste en obtener un puntero que apunte la zona de memoria, para poder escribir o leer en ella. Se declara en el código un puntero al tipo que se sepa que va a haber en la zona de memoria (una estructura, un *array*, tipos simples, etc.) y se utiliza la función `char * shmatt (int, char *, int)`:

- El primer parámetro es el identificador de la memoria obtenido en el paso anterior.
- Los otros dos bastará rellenarlos con ceros.
- El puntero devuelto es de tipo `char *`. Se debe un "cast" al tipo que se quiera, por ejemplo, `(mi_estructura *)shmatt (...)`; Esta función lo que en realidad hace, además de dar el puntero, es asociar la memoria compartida a la zona de datos del programa, por lo que es necesario llamarla sólo una vez en cada proceso. Si se quieren más punteros a la zona de memoria, bastará con igualarlos al que ya se tiene.

5) Ya se está en condiciones de utilizar la memoria. Cualquier cosa que se escriba en el contenido del puntero, se escribirá en la zona de memoria compartida y será accesible para los demás procesos.

6) Una vez terminada de usar la memoria, debe ser liberada. Para ello se utilizan las funciones `intshmtdt (char *)` e `intshmctl (int, int, structshmids *)`. La primera función desasocia la memoria compartida de la zona de datos del programa. Basta pasarle el puntero que se tiene a la zona de memoria compartida y llamarla una vez por proceso. La segunda función destruye realmente la zona de memoria compartida. Hay que pasarle el identificador de memoria obtenido con `shmget()`, un *flag* que indique que se quiere destruirla `IPC_RMID`, y un tercer parámetro al que bastará con pasarle un NULL. A esta función sólo debe llamarla uno de los procesos.

Hasta ahora se indicado lo mínimo y más básico para poder utilizar memoria compartida. Hay más funciones que permiten hacer más cosas con la memoria compartida. Además, para un uso más "serio" y "correcto", sería necesario añadir algo como semáforos.

Por ejemplo, si los datos que se desean compartir son muy grandes, es bastante probable que mientras un proceso intenta escribir en la memoria, otro esté leyendo. Si lo intentan de forma totalmente descontrolada y a la vez, los datos que leen pueden ser incoherentes (hay otro que está escribiéndolos en ese momento y todavía no ha terminado).

Para evitar esto existen los semáforos. Se pondría un semáforo para acceder a la memoria, de forma que cuando un proceso lo está haciendo, el semáforo se pone "rojo" y ningún otro proceso puede acceder a ella. Cuando el proceso termina, el semáforo se pone "verde" y ya podría acceder otro proceso.

Comandos útiles en Linux

Unos comandos de Linux que pueden resultar de utilidad son **ipcs** y **ipcrm**.

ipcs da un listado de recursos compartidos que están creados en ese momento, es decir, listado de memorias compartidas como las que se han tratado en esta página, de semáforos y de colas.

ipcrm permite eliminar algunos de estos recursos. Si se para el programa con un Ctrl-C o simplemente sale de forma anormal, el recurso (la memoria compartida) no se libera y queda en el sistema. La forma de borrarla sería con este comando.

Es bastante normal que mientras se desarrolla y depura el programa, el mismo se caiga, sea abortado, etc. El número de memorias compartidas que se puede crear está limitado en Linux, así que a la cuarta o quinta prueba se empezarán a obtener errores de que no se pueden crear las memorias. **ipcrm** permite eliminar las memorias que se han quedado "pendientes" en las pruebas.

Código de memoria compartida

Proceso 1

```
#include <sys/shm.h>
#include <unistd.h>
#include <stdio.h>
```

```

#include <stdlib.h>
#include <sys/ipc.h>

#define CANTIDAD 10
#define CANTIDAD_TOTAL 10
#define DESDE 0
#define HASTA 25

#define CLAVE_BASE 33

typedef struct tipo_dato dato;

struct tipo_dato
{
    int numero;
    char letra;
};

key_t creo_clave(int r_clave)
{
    // Igual que en cualquier recurso compartido (memoria compartida, semáforos
    // o colas) se obtiene una clave a partir de un fichero existente cualquiera
    // y de un entero cualquiera. Todos los procesos que quieran compartir esta
    // memoria, deben usar el mismo fichero y el mismo entero.
    key_t clave;
    clave = ftok ("/bin/ls", r_clave);
    if (clave == (key_t)-1)
    {
        printf("No puedo conseguir clave para memoria compartida\n");
        exit(0);
    }
    return clave;
}

void* creo_memoria(int size, int* r_id_memoria, int clave_base)
{
    void* ptr_memoria;
    int id_memoria;
    id_memoria = shmget (creo_clave(clave_base), size, 0777 | IPC_CREAT);

    if (id_memoria == -1)

```

```

{
    printf("No consigo id para memoria compartida\n");
    exit (0);
}

ptr_memoria = (void *)shmat (id_memoria, (char *)0, 0);

if (ptr_memoria == NULL)
{
    printf("No consigo memoria compartida\n");

    exit (0);
}
*r_id_memoria = id_memoria;
return ptr_memoria;
}

int main()
{

    int id_memoria;
    dato *memoria = NULL;
    int i, aleatorio;

    memoria = (dato*)creo_memoria(sizeof(dato)*CANTIDAD, &id_memoria, CLAVE_
BASE);

    srand(time(NULL));

    while(1)
    {
        printf("Escribimos en la memoria\n");
        for (i=0; i<CANTIDAD; i++)
        {
            aleatorio = rand()%(HASTA-DESDE)+DESDE;
            memoria[i].numero = aleatorio ;
            memoria[i].letra = 0x41 + aleatorio ;

```

```

        printf("Escrito %d %c\n", memoria[i].numero, memoria[i].letra);
        sleep (1);
    }
    printf("Dejamos de escribir en la memoria\n");
    sleep (10);
}
//
// Terminada de usar la memoria compartida, la liberamos.
//
shmdt ((char *)memoria);
shmctl (id_memoria, IPC_RMID, (struct shmid_ds *)NULL);
return 0;
}

```

Proceso 2:

```

#include <sys/shm.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#define CANTIDAD 10
#define CANTIDAD_TOTAL 10
#define CLAVE_BASE 33

typedef struct tipo_dato dato;

struct tipo_dato
{
    int numero;
    char letra;
};

key_t creo_clave(int r_clave)
{
    key_t clave;
    clave = ftok ("/bin/ls", r_clave);
    if (clave == (key_t)-1)
    {
        printf("No puedo conseguir clave para memoria compartida\n");
        exit(0);
    }
    return clave;
}

```

```

}

void* creo_memoria(int size, int* r_id_memoria, int clave_base)
{
    void* ptr_memoria;
    int id_memoria;
    id_memoria = shmget (creo_clave(clave_base), size, 0777 | IPC_CREAT);

    if (id_memoria == -1)
    {

        printf("No consigo id para memoria compartida\n");
        exit (0);
    }

    ptr_memoria = (void *)shmat (id_memoria, (char *)0, 0);

    if (ptr_memoria == NULL)
    {
        printf("No consigo memoria compartida\n");

        exit (0);
    }
    *r_id_memoria = id_memoria;
    return ptr_memoria;
}

int main()
{
    int id_memoria;
    dato *memoria = NULL;
    int i,j;

    memoria = (dato*)creo_memoria(sizeof(dato)*CANTIDAD, &id_memoria, CLAVE_
BASE);

    // Vamos leyendo el valor de la memoria con esperas de un segundo
    // y mostramos en pantalla dicho valor. Debería ir cambiando según
    // p1 lo va modificando.
    //
    while(1)
    {

```

```

    printf("leemos la memoria\n");
    for (i=0; i<CANTIDAD; i++)
    {
        printf("Leido %d %c\n", memoria[i].numero, memoria[i].letra);
    }
    printf("Dejamos de leer la memoria\n");
    sleep (1);
}
// Terminada de usar la memoria compartida, la liberamos.
shmdt ((char *)memoria);
shmctl (id_memoria, IPC_RMID, (struct shmid_ds *)NULL);
return 0;
}

```